

From experiment logs to defensible claims: an artifact audit of coding-agent-assisted small-model research

Abstract

Context: Coding agents can help construct experimental software, training data, evaluators, and research narratives. Agreement among those artifacts may conceal a shared error rather than provide independent confirmation. **Objective:** We investigate how artifact-level auditing changes the claims supported by one agent-assisted machine-learning project. **Method:** An exploratory retrospective case study reconstructs ten archived fine-tuning arms from 7,050 item records, resolves model identity from nested manifests, joins evaluation items across versions, checks comparator behavior, and inspects the evidence behind semantic judgments and mechanism claims. **Results:** All original normalized exact outcome labels are reproducible, but several interpretations require revision. An arm described as a 17B model is actually Qwen3-1.7B; a later dataset version replaces 50 of 705 evaluation items; and 833 retained semantic-judge inputs lack the decisions needed to reproduce the reported semantic aggregates. A positive abstraction result survives: matches rise from 379/705 to 440/705, with procedural execution errors falling from 63/480 to 13/480. A target-decomposition change produces the opposite direction. **Conclusion:** The case supports a claim-centered audit that separates identity, population, measurement, and mechanism evidence. It does not estimate coding-agent productivity or compare agent-assisted and human-only research. We provide executable reconstruction, conservative diagnostic parsers, and a prospective audit protocol for future vocabulary-discovery experiments.

Keywords: empirical software engineering; coding agents; reproducibility; case study; research software; evidence provenance

1. Introduction

Research software increasingly includes work performed with coding agents. In a machine-learning project, the same workflow can produce a data generator, a reference solver, a validator, a training launcher, an evaluation script, and a polished account of the results. The resulting consistency is attractive: identifiers recur across files, tables agree with summaries, and automated checks pass. Yet consistency can arise because the artifacts share an incorrect assumption.

The empirical problem is to determine what that collection of artifacts actually supports. A training run may be real while its model label is misleading. An accuracy count may be correct while the compared populations differ. A successful output may be genuine while the proposed explanation for its success is unsupported by the emitted program. These are different failure modes and require different audits.

We study one repository that fine-tunes small language models to compile word problems into SOP Lang circuits. The research was conducted with coding-agent assistance, including implementation, experiment tooling, analysis, and writing. The system provides unusually inspectable evidence because each evaluated output is a program and the archive preserves per-item outcomes. It also contains drafts making claims about model size, abstract operations, and semantic accuracy. This combination permits an artifact-based examination of the path from run records to publication claims.

The study asks three questions. RQ1: Which claims can be reconstructed from surviving artifacts? RQ2: Which discrepancies arise between reproducible measurements and their interpretation? RQ3: Which audit procedures preserve useful findings while restricting unsupported conclusions? The intended contribution is an empirically grounded audit method, not a claim that coding agents caused every defect or that human-only research would have avoided them.

2. Background and analytical perspective

Runeson and Höst describe case-study research as appropriate for examining software-engineering phenomena in their context (Runeson & Höst, 2009). Our context is one evolving research repository, and our evidence is its technical archive. There are no interviews, randomized developer assignments, or measured productivity outcomes. The case can identify concrete risks and useful audit operations; it cannot estimate their prevalence across teams.

Sandve and colleagues emphasize preserving the information needed to reconstruct computational results (Sandve et al., 2013). That requirement motivates executable extraction from raw item records rather than copying tables from reports. However, a reproducible number can answer the wrong question. Kapoor and Narayanan's analysis of leakage illustrates how failures in experimental design can undermine conclusions even when code runs successfully (Kapoor & Narayanan, 2023). In this case we distinguish adaptive reuse of a development benchmark from proven direct training contamination; the former is documented, while the latter is not established by the audit.

Raji and colleagues frame internal algorithmic auditing as a process extending across the development lifecycle (Raji et al., 2020). We adapt the idea to research claims. The audit object is a chain connecting a statement in a manuscript to the model identity, data population, comparator, item outcomes, and proposed explanation. A broken link can narrow a claim without invalidating every other link.

Messeri and Crockett discuss how AI use in science can create an illusion of understanding (Messeri & Crockett, 2024). This motivates examining whether executable checks and fluent interpretation were being treated as interchangeable evidence. Their argument is conceptual; it does not establish that coding agents caused the specific errors observed here. Our evidence for those errors comes from the repository itself.

3. Case and research method

3.1 System and case boundary

The case system trains a small model to emit a circuit from a problem statement. Named wires expose literal data, JavaScript computation, and specialized commands such as graph reachability, aggregation, and fraction reduction. A runtime checks structure, schedules dependencies, executes operations, and records outcomes. Its purpose resembles the computation delegation used in PAL (Gao et al., 2023), with a project-specific target language and training pipeline.

The archive analyzed here contains ten selected arms, spanning an early vocabulary change and a later curriculum series. Each arm contributes a 705-item evaluation file. The analysis includes 7,050 item records, the corresponding metric summaries, checkpoint selections, evaluation manifests, available training manifests, base-model metadata, relevant evaluator source, and the earlier article materials. The selection is purposive: these are the runs needed to examine the principal claims in the drafts. It is not a census of every experiment in the repository.

The audit snapshot is identified by repository revision 922debb5242e10e3e9c4c7d8b7f85b927d09d54e for the pre-audit tracked source, together with SHA-256 hashes for the experimental files read. Some experimental artifacts are outside the tracked source history, making their content hashes necessary. The companion evidence manifest records those paths and hashes. No training or model inference was repeated, and existing artifacts were not rewritten to agree with the new analysis.

3.2 Evidence sources and audit operations

Table 1 defines the relationship between evidence and claim. We treat an identifier as a pointer to evidence, not as authoritative metadata. This is particularly important for model names and data-version labels, which can survive after an underlying experimental decision changes.

Table 1. Claim classes and the audit operation needed to support them.

Claim class	Evidence inspected	Audit operation
Model identity	Nested base manifest, repository and revision fields	Resolve the actual base instead of parsing the arm label
Population identity	Item identifiers, oracles, plan fingerprints	Join across arms and count replacements and changes
Outcome count	Per-item classes and evaluator code	Recount classes and rerun the original comparator
Semantic equivalence	Individual decisions or a specified parser	Retain decisions; reject unsupported aggregate reconstruction
Mechanism	Emitted programs and command declarations	Check whether the proposed operation was actually used
Causal attribution	Training configuration, seeds, interventions	State confounding and missing control information

The extraction script checks that each arm has 705 unique identifiers, partitions all records into known outcome classes, and reproduces the stored metrics. It reapplies the original normalized exact comparator to each completed answer. It then joins selected arm pairs and records changed identifiers, oracle strings, and plan fingerprints. Finally, it inspects generated command declarations and the surviving semantic-judge artifacts.

The original comparator normalizes Unicode, case, whitespace, and selected punctuation before equality. A mismatch under this rule is evidence of failed normalized textual agreement, not necessarily failed computation. We retain that distinction when interpreting the outcome ladder.

3.3 Retrospective semantic diagnostics

To investigate specific answer-format discrepancies without inventing missing judgments, we implement two restricted parsers. One accepts complete coalition/count tuple lists, normalizes set ordering, and rejects duplicates. The other accepts complete dependency-join sentences expressing a duration and a feasibility verdict through a fixed set of phrases. It recognizes the oracle’s known explanatory suffix. Both require the whole answer to fit the declared grammar.

The parser outputs are match, mismatch, outside grammar, or execution failure. Outside grammar is an explicit abstention. We preserve each decision and its parsed values in JSONL files. These diagnostics were designed after inspecting the archive and are therefore exploratory. They neither reproduce the missing historical judge decisions nor establish a general replacement score.

3.4 Claim assessment and reflexivity

We assign conclusions to five evidence states: reproduced from item records, corroborated by additional artifacts, historical-only, proposed, or unresolved. This classification separates a missing artifact from a negative result. For example, missing judge decisions make a semantic aggregate non-reproducible from the available archive; they do not prove every underlying judgment was wrong.

The audit itself used coding-agent assistance. It is consequently not independent peer review or blinded human adjudication. Executable assertions, retained inputs, explicit abstention, and inspection of source contracts reduce specific risks, but cannot remove every shared assumption. We document that boundary because calling a second automated pass “independent validation” would repeat the problem under study.

4. Findings

4.1 RQ1: the primary outcome counts are reproducible

Every selected arm contributes 705 unique records, and reconstructed class totals agree with the archived metrics. Reapplying the original comparator yields no disagreements with stored match/mismatch labels. Syntax and graph acceptance is 705/705 for every arm. This establishes a stable descriptive base for the study.

Table 2 shows the reconstructed counts. Completed mismatches and execution failures are kept separate because they call for different remedies. A comparator revision can affect the former, while it cannot turn a program that failed to execute into a successful answer.

Table 2. Audited outcome counts for the ten selected arms, each evaluated on 705 items.

Arm and data	Normalized exact match	Completed mismatch	Execution error
exp-014, dv2	379	125	201
exp-016, dv3	440	90	175
exp-017, dv3	442	111	152
exp-021, dv7	460	188	57
exp-022, dv8	448	122	135
exp-023, dv9	428	185	92
exp-024, dv11	439	154	112
exp-025, dv12	432	137	136
exp-026, dv13	361	130	214
exp-027, dv13	421	100	184

The counts also preserve useful negative evidence. The dv7-to-dv8 target-decomposition change reduces matches from 460/705 to 448/705 and increases execution errors from 57/705 to 135/705 on unchanged identifiers and oracle strings. A persuasive narrative about modularity must account for this result rather than treating additional structure as inherently beneficial.

4.2 RQ2: identity and population errors change the comparison

The first material discrepancy concerns model identity. The experiment directory `exp-017-qwen3-17b` had been interpreted as a 17B model. Nested base-model metadata identifies Qwen3-1.7B and the same pinned revision used by the later 1.7B arms. The corresponding “small model beats 17B” claim is unsupported. Correcting the label preserves the observed curriculum difference while removing the scale comparison.

The second concerns the evaluation population. Dv7 and dv13 each have 705 items, but only 655 identifiers are shared; 50 are removed and 50 introduced. Among the shared identifiers, 100 oracle strings and 147 plan fingerprints change. One task family moves into training and another takes its evaluation place. Aggregate scores with the same denominator therefore do not identify a fixed-test regression.

The audit also narrows the smaller-model comparison. Exp-026 uses Qwen2.5-Coder-0.5B-Instruct; exp-027 uses Qwen3-1.7B. Their dv13 items, oracles, and plans match, and their scores are 361/705 and 421/705. Size changes together with model family and pretraining. A claim about those two trained systems is supported; a universal minimum size for reasoning is not.

4.3 RQ2: judgment records and mechanism evidence are incomplete

An earlier report gives a broad semantic score for several arms. The retained judge shards contain 833 unique inputs with keys, oracle texts, and model answers. They do not contain the corresponding verdicts or rationales. Four disjoint input shards also provide no evidence of inter-rater agreement. The broad semantic aggregates therefore remain historical-only.

The new restricted diagnostics recover a narrower, inspectable finding. For exp-027 coalitions, 20/20 answers match as tuple sets, compared with 5/20 under normalized exact text. For exp-021 dependency joins, 64/100 match under the declared duration/verdict grammar, compared with 0/100 originally. Another 26 cases are outside that grammar, two are parsed disagreements, and eight are execution failures. The method makes its refusal to classify visible, rather than converting every unrecognized phrase into a judgment.

Mechanism inspection produces a separate correction. The container-oriented dv7 curriculum achieves 20/20 coalition matches where exp-017 achieves 0/20. Yet none of those 20 dv7 outputs declares a container-family command. The measured improvement survives, but a claim that container execution caused it does not. The subset contains one coalition plan, which also rules out treating the result as complete coverage of its source book.

Table 3. Consequential corrections and their supported replacements.

Earlier interpretation	Audit finding	Supported replacement
A 1.7B model beats a 17B model	Both named arms use Qwen3-1.7B	A later curriculum improves selected outcomes for the same base
Equal denominators imply the same test	50 identifiers replaced between dv7 and dv13	Compare shared populations and disclose oracle changes
Semantic totals are independently verified	Judge inputs survive, decisions do not	Retain exact totals and separately report restricted diagnostics
Containers explain coalition success	No container declarations in the 20 outputs	Report a family-specific curriculum association
The 0.5B result establishes a size floor	Model family and pretraining also change	Describe the two systems without a universal threshold

4.4 RQ3: a positive result survives stricter claim boundaries

Auditing does not reduce the case to a catalogue of mistakes. The early vocabulary transition remains a substantive finding. Qwen2.5-Coder-1.5B-Instruct increases matches from 379/705 in exp-014 to 440/705 in exp-016. Identifiers and oracle strings are unchanged. The paired records contain 377 joint matches, 63 matches exclusive to exp-016, two exclusive to exp-014, and 263 joint non-matches.

Within the 480 procedural items, matches rise from 377 to 440 while execution errors fall from 63 to 13. The revised system introduces graph, aggregation, and fraction commands. These operations replace recurring implementation code with narrower contracts. Forty procedural outputs directly use at least one of them. This is encouraging evidence for vocabulary design as a research variable, while incomplete exp-014 training metadata and one run per condition prevent a clean causal estimate.

The distinction is useful for empirical software engineering. A defensible claim need not be either a universal causal law or an anecdote without evidential value. Here, reconstructed paired outcomes establish a development signal, source inspection supplies a plausible mechanism, and missing controls define the next experiment.

Figure 1 summarizes the audit chain that connects an observed outcome to a bounded claim.

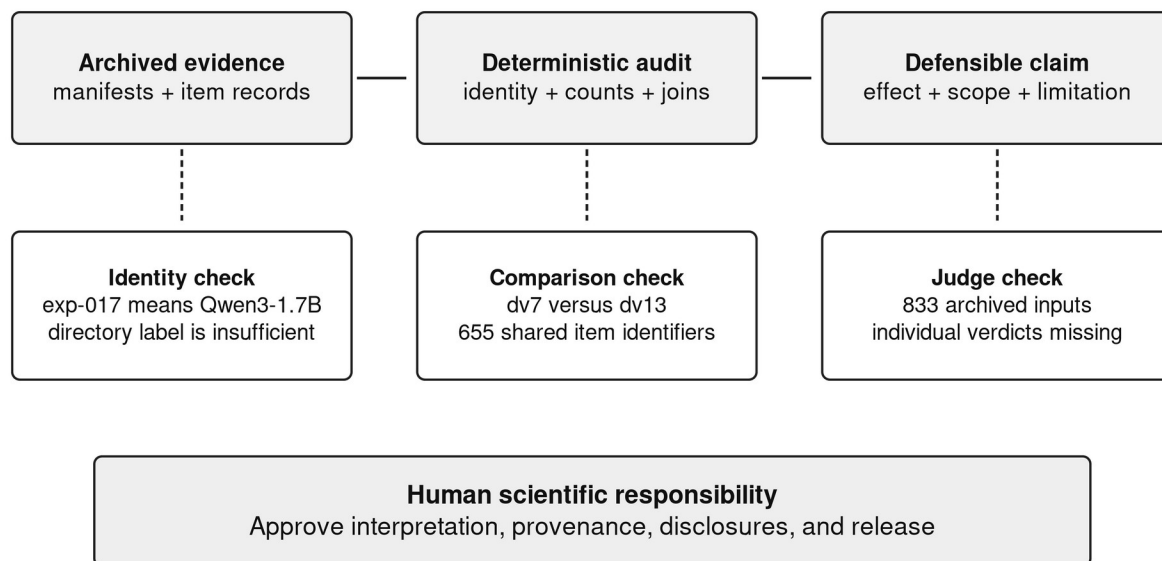


Fig. 1 The claim-centered audit used in the case. A reproduced aggregate is one part of the chain; human scientific responsibility remains necessary at the publication boundary

5. Discussion

5.1 Consistency is a weak substitute for independent evidence

Several original summaries were numerically consistent with stored metrics. Their problems lay in what those metrics were taken to represent. A model-size label changed the comparator class; a constant sample size concealed a changed population; a curriculum name became a mechanism claim. A workflow that checks only arithmetic consistency would miss all three.

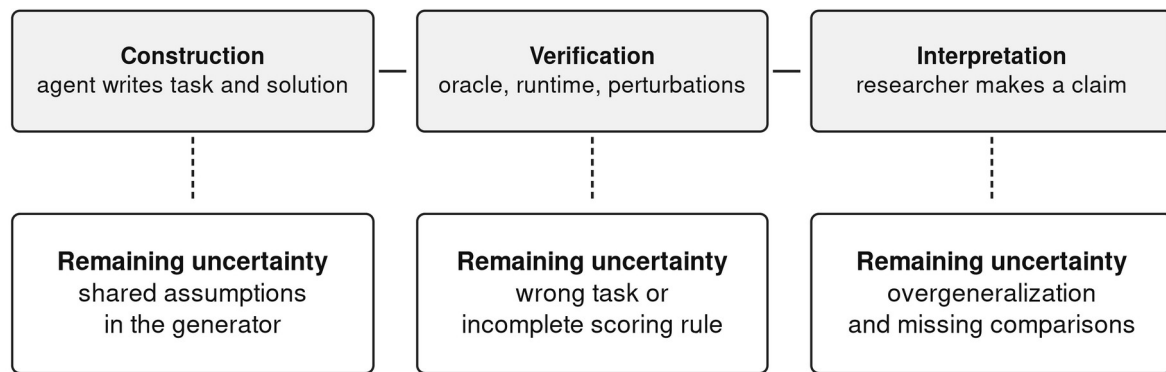
This suggests separating four records in agent-assisted research. The run record identifies the system and its inputs. The measurement record defines the observation procedure. The interpretation record connects observations to a claim. The release record states what another researcher can actually inspect. These records can be linked, but one should not silently supply missing content for another.

The missing semantic decisions illustrate why this separation matters. A text summary can truthfully preserve what a previous analysis reported, yet still be insufficient to reproduce it. An auditor should label the evidential gap rather than either accepting the aggregate or inventing new decisions under its name. Fresh diagnostics are useful when they are identified as fresh analyses.

5.2 Data rigor needs differentiated checks

The training pipeline made a serious attempt to check accepted examples. Reference computations, executable circuits, assertions, and input perturbations address concrete failure modes. These controls are more informative than accepting generated examples solely because their prose looks plausible. However, they do not create independence automatically. A reference parse and its circuit can agree because both misread the same sentence.

Figure 2 locates the uncertainties that agreement between generated artifacts can leave unresolved.



Agreement inside a pipeline does not establish independence.

Keep evidence that can challenge each stage.

Fig. 2 Why additional checks need different evidential roles. Repeating a shared assumption in a generator, test, and report does not establish independent corroboration

Gebru and colleagues' datasheet approach motivates documenting composition, construction, and intended use (Gebru et al., 2021). Mitchell and colleagues' model cards motivate identifying the trained model and its evaluation context (Mitchell et al., 2019). Applied to this case, those practices should include the benchmark's repeated developmental use, family concentration, changed oracles, and unsupported semantic aggregates. A card that lists only a model name and an accuracy number would omit the facts that materially affect interpretation.

The archive also demonstrates a limit of claims about generalization. In dv7, the 705 items represent 28 plan fingerprints, and several book-derived subsets repeat a single plan. The label "unseen family" requires a version-specific definition of what was excluded and when it was inspected. Repeated use of the nominal holdout for design converts it into development evidence, consistent with the concerns about adaptive analysis articulated by Dwork and colleagues (Dwork et al., 2015).

5.3 Implications for future wire-discovery studies

The next study should attach an audit specification to the experiment plan. Each proposed wire should identify the error class it addresses, excluded uses, reference implementation, and transfer families that remain sealed. The model, tokenizer, training export, token budget, checkpoint rule, comparator, and inference settings should be pinned before comparison. Multiple seeds would permit estimating variability across training runs.

The audit should also require command-use evidence before attributing a gain to a command. A curriculum can affect generated JavaScript even when the model never calls the new operation. Distinguishing direct use from indirect training effects makes a stronger experiment possible. It prevents an attractive architectural story from outrunning the observed programs.

Finally, automated judgment should preserve every decision, its input, the judge identity and version, and the procedure for disagreement or abstention. Where structured answers are feasible, a task-defined comparator is preferable to a retrospective interpretation of prose. Human review should target ambiguous specifications and informative disagreements, rather than merely approve aggregate tables.

6. Threats to validity

Construct validity is limited by normalized exact matching, synthetic task definitions, and plan fingerprints that approximate structure. The restricted parsers address two concrete formatting issues but do not measure general

semantic correctness. Internal validity is limited by non-randomized developmental interventions, checkpoint selection, changing target lengths, one seed per condition, and missing exp-014 training metadata.

External validity is limited to one repository, a small set of model families, and generated word problems. The study does not measure deployment behavior, software-team productivity, or the rate of agent-induced errors across projects. There is no human-only comparison, so agent assistance cannot be identified as the cause of the observed discrepancies.

Reliability is improved by executable extraction, source hashes, item-level diagnostics, and preserved original artifacts. It remains limited by unavailable historical judgment decisions and the absence of a fresh training replication. The audit was assisted by the same class of tools being examined and should be reviewed independently before claims are treated as externally validated.

7. Conclusion

The case shows how an experiment archive can support reliable counts and unreliable interpretations at the same time. Checking identity, population, measurement, and mechanism changes the strongest claims while preserving a useful positive result about abstract wires and a negative result about target decomposition. A claim-centered audit gives coding-agent-assisted research a concrete path from executable artifacts to bounded scientific conclusions. Its value is demonstrated here through corrected evidence, not through an unmeasured claim of research automation or productivity.

Data availability and declarations

The companion artifact, Online Resource 1, provides reconstruction scripts, source hashes, paired comparison tables, restricted parser decisions, and a correction ledger. Original evaluation records and run manifests remain the primary evidence. Human participant data were not collected for this artifact study. Coding-agent assistance is described in Sections 1 and 3.4; it does not imply agent authorship. Contributor identities, affiliations, funding, competing interests, and public archive details require author confirmation before submission.

References

- Dwork, C., Feldman, V., Hardt, M., Pitassi, T., Reingold, O., & Roth, A. (2015). The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248), 636-638. <https://doi.org/10.1126/science.aaa9375>
- Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., & Neubig, G. (2023). PAL: Program-aided Language Models. *Proceedings of Machine Learning Research*, 202, 10764-10799. <https://proceedings.mlr.press/v202/gao23f.html>
- Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé, H., III, & Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12), 86-92. <https://doi.org/10.1145/3458723>
- Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 100804. <https://doi.org/10.1016/j.patter.2023.100804>
- Messeri, L., & Crockett, M. J. (2024). Artificial intelligence and illusions of understanding in scientific research. *Nature*, 627(8002), 49-58. <https://doi.org/10.1038/s41586-024-07146-0>
- Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220-229. <https://doi.org/10.1145/3287560.3287596>
- Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., & Barnes, P. (2020). Closing the AI accountability gap: defining an end-to-end framework for internal algorithmic auditing. *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 33-44. <https://doi.org/10.1145/3351095.3372873>

- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131-164. <https://doi.org/10.1007/s10664-008-9102-8>
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten Simple Rules for Reproducible Computational Research. *PLoS Computational Biology*, 9(10), e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>